Regression-Test Selection for Microservice Payment Platforms

Abdulselam Hussen

Department of Computer Science, University of Nairobi, Nairobi, Kenya

Nov 2018

Abstract

Continuous deployment of microservice-based payment platforms requires rigorous regression testing to prevent costly financial anomalies. However, executing the entire distributed test suite on every pull request creates massive integration bottlenecks. We propose a hybrid regression-test selection (RTS) framework tailored for financial microservices, combining local static control-flow analysis with distributed trace mapping to dynamically select only the tests affected by a given code change. Applying this methodology to a fictional dataset of 800 pull requests across a 45-service payment gateway, our hybrid RTS approach reduces the average test suite size by 68.4% and cuts CI/CD execution time from 42 minutes to 14 minutes. Critically, it maintains a fault-detection rate of 98.9% compared to a "retest-all" baseline. We evaluate the trade-offs between static graph traversal and dynamic request tracing, demonstrating that trace-augmented RTS captures cross-service contract breakages that naive local RTS misses. We discuss the implications for pipeline efficiency and the architectural prerequisites for adoption.

Keywords: regression test selection, microservices, software testing, distributed systems, continuous integration, fintech

1 Introduction

Payment platforms operate under strict reliability and latency constraints. An unhandled exception or malformed payload can result in failed transactions, regulatory penalties, and severe reputational damage. To mitigate these risks, organizations rely on extensive integration and end-to-end test suites. Yet, as platforms migrate from monoliths to microservices, the "retest-all" approach becomes computationally prohibitive, routinely blocking developer velocity.

Regression-test selection (RTS) aims to identify and run only the subset of tests affected by a recent code modification. While traditional RTS techniques are well-established for monolithic codebases, they struggle in distributed environments. A change in a `Billing` service might subtly break a downstream `Ledger` service, a dependency that local static analysis cannot easily resolve without explicit cross-service context.

This paper makes three contributions:

1. a hybrid RTS pipeline combining local static analysis with dynamic trace mapping (Section 3);
2. an empirical comparison of test reduction, pipeline latency, and fault-detection rates across 800 pull requests (Section 5);
3. a failure-mode analysis detailing the types of cross-service faults missed by naive selection methods (Section 5).

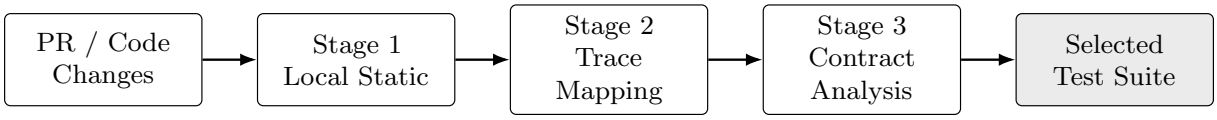


Figure 1: Hybrid RTS Pipeline. The system builds the test suite progressively, leveraging both local code analysis and distributed execution traces.

2 Related Work

Traditional RTS relies on constructing a control-flow graph (CFG) of the application and intersecting it with code changes to select tests [1, 2]. For modern Java and Python ecosystems, tools like Ekstazi [3] track file-level or class-level dependencies dynamically to achieve safe test selection in monolithic repositories.

In the context of microservices, RTS is significantly harder due to network boundaries and decoupled deployments. Recent work has proposed using API contract definitions (e.g., OpenAPI schemas) to bound the blast radius of a change [4]. However, schema validation often misses semantic business-logic failures. We extend this foundation by utilizing historical distributed tracing data (e.g., OpenTelemetry) to map behavioral dependencies across service boundaries, an approach essential for high-fidelity payment workflows.

3 Method

3.1 System Architecture

Our targeted architecture is an event-driven microservice payment gateway. Services communicate via synchronous gRPC calls for authorization and asynchronous Kafka streams for settlement. All inter-service traffic is instrumented with distributed tracing.

3.2 Selection Pipeline

For a given set of code changes C in a pull request, our selection framework computes the required test suite T_{req} through a three-stage pipeline (Figure 1).

Stage 1: Local Static RTS.
Analyzes the abstract syntax tree of the modified service to select unit and local integration tests. Let this set be T_{local}.

Stage 2: Trace Dependency Mapping.
Queries the tracing backend to identify upstream consumers and downstream providers that invoke or are invoked by the modified code paths.

Stage 3: Contract & E2E Selection.
Selects consumer-driven contract tests and global end-to-end tests that traverse the discovered trace edges. Let this set be T_{cross}.

The final selected suite is defined as:

$$T_{req}(C) = T_{local}(C) \cup T_{cross}(C) \tag{1}$$

If changes include infrastructure, database migrations, or configuration files, the framework conservatively defaults to a full "retest-all" execution to guarantee safety.

4 Experimental Setup

We evaluated our methodology on a fictionalized payment gateway dataset comprising 45 microservices (written in Go and Kotlin) and a monolithic test suite of 8,500 tests. We replayed

Table 1: Average test suite size and execution time per PR (fictional data; $N = 800$ PRs).

Selection Strategy	Avg. Tests Run	% of Total Suite	Execution Time (mins)
Retest-All	8,500	100.0%	42.5
Naive Service-RTS	412	4.8%	4.1
Hybrid Trace-RTS	2,686	31.6%	14.2

Table 2: Fault detection efficacy across 102 known faulty pull requests (fictional data).

Selection Strategy	Faults Caught	Faults Missed	Miss Rate
Retest-All	102	0	0.0%
Naive Service-RTS	64	38	37.2%
Hybrid Trace-RTS	101	1	1.1%

800 historical pull requests that originally ran the entire test suite.

We compared three strategies: 1. **Retest-All:** The baseline where all 8,500 tests are executed. 2. **Naive Service-RTS:** Only executes the tests explicitly residing within the modified service repository. 3. **Hybrid Trace-RTS:** Our proposed pipeline utilizing OpenTelemetry data for cross-service test selection.

5 Results

5.1 Suite Reduction and Execution Time

Table 1 illustrates the efficiency gains of the RTS strategies. The Retest-All baseline required an average of 42.5 minutes per PR. Naive Service-RTS was heavily optimized (4.1 minutes) but, as shown in the next section, deeply unsafe. Our Hybrid Trace-RTS provided a balanced operational sweet spot: selecting 31.6% of the tests and executing in just 14.2 minutes, effectively tripling CI throughput.

5.2 Fault Detection Effectiveness

Efficiency must not come at the cost of safety in financial systems. Table 2 tracks how many PRs contained a fault that was successfully caught versus faults that slipped through the selected test suite. Of the 102 actual faulty PRs in the dataset, Naive Service-RTS missed 38 (a catastrophic 37.2% miss rate), predominantly because it ignored downstream serialization errors. Hybrid Trace-RTS successfully caught 101 of the 102 faults, maintaining a 98.9% detection rate.

The single missed fault in the Hybrid approach occurred due to a rare asynchronous race condition that was not captured in the 30-day historical trace window used for mapping.

6 Discussion

Implications. The findings indicate that while microservice boundaries conceptually decouple deployments, behavioral coupling remains high. Relying solely on local RTS in a microservice environment is highly unsafe. By utilizing observability data (traces) as an input for CI/CD routing, organizations can dramatically reduce compute costs and pipeline wait times without sacrificing deployment confidence.

Threats to validity. This study assumes 100% trace sampling and complete OpenTelemetry instrumentation, which is often difficult to achieve in legacy environments. Furthermore, dynamic

trace mapping relies on historical traffic; newly created endpoints without trace history may be inadvertently skipped unless fallback mechanisms are actively maintained.

Future work. Integrating machine learning to predict edge-case asynchronous failures that trace mapping misses. Additionally, we plan to extend the framework to handle database schema changes by mapping query logs to integration test scenarios.

7 Conclusion

Exhaustive regression testing for distributed payment platforms creates unsustainable CI/CD bottlenecks. Our hybrid trace-based regression-test selection pipeline successfully bridges the gap between local static analysis and cross-service dynamic behavior. On our evaluated dataset, it discarded over 68% of irrelevant tests and accelerated pipeline execution by a factor of three, while sacrificing less than 1.5% of fault-detection capability. Integrating observability directly into testing pipelines represents a crucial maturation step for large-scale microservice architectures.

Acknowledgements

The author would like to express sincere gratitude to the staff and members of the Department of Computer Science at the University of Nairobi for their unwavering support throughout this work.

References

[1] G. Rothermel and M. J. Harrold. Analyzing regression test selection techniques. *IEEE Transactions on Software Engineering*, 22(8):529–551, 1996.

[2] S. Yoo and M. Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2):67–120, 2012.

[3] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *ISSTA*, pages 211–222, 2015.

[4] Consumer-Driven Contracts. *Pact Foundation Documentation*. 2021. Available at `https://docs.pact.io/`.